COMPANY Details

Voxel is a **San Francisco AI/computer vision company** focused on workplace safety, operations, and risk in industrial environments. Their platform uses **existing camera infrastructure** and turns video into actionable safety/operations insights, instead of requiring customers to install totally new hardware.

Their core pitch is: **move safety from reactive to proactive**. Instead of only reviewing incidents after someone gets hurt, Voxel detects risks like PPE issues, forklift/vehicle hazards, ergonomics problems, blocked exits, spills, unsafe behaviors, and operational bottlenecks before they become incidents.

Their main industries are **warehousing, manufacturing, retail distribution, logistics/supply chain, and insurance/risk management**. Their public customer examples include **Americold, Port of Virginia, Piston Automotive, NSG, Carlex Glass, MSI, Dick's Sporting Goods, AGI, and Berry Global**.

They raised **$44M Series B in June 2025**, led by NewRoad Capital Partners, bringing official reported funding at that time to **$61M**. The funding was aimed at accelerating R&D, improving real-time computer vision models, and expanding data insights.

Their website currently highlights **$61M raised, 90 team members, 250+ deployed sites worldwide, and 65% average reduction in workers' compensation claims**. Their leadership includes **Vernon O'Donnell, CEO**, and **Bryan O'Sullivan, CTO**.

Recent news you can mention: in **April 2026**, Voxel launched **Risk Insights**, a beta feature that gives site managers personalized AI-powered daily safety snapshots through the portal, email, and SMS. It explains what is happening, where/when it is happening, and why, using video-backed root-cause analysis.

Another strong recent hook: Voxel announced an **expanded San Francisco headquarters on May 27, 2026**, specifically tied to growth and hiring.

Another hook: on **June 9, 2026**, Voxel announced that CEO Vernon O'Donnell would keynote Reuters Supply Chain USA 2026 on **"Ground-Level Intelligence: How Applied AI is Redefining Supply Chain Performance."** The message was that real-time camera data and computer vision can give companies continuous visibility into what is actually happening on the floor.

INTRO / Tell me about yourself:
Hi, my name is Sahaj Bhadja. I recently completed my MS in Computer Science at USC, and my background is mainly in full-stack application engineering — building React dashboards, backend APIs, authentication and authorization flows, data models, and cloud-based systems.

Most recently at USC Dornsife, I worked as a Software Engineer where I built backend services and dashboard features for an internal platform used by 500+ users. My work involved Azure SQL, row-level authorization, async job handling, API development, and improving dashboard performance.

Before that, at Grad8, I worked on an AI grading platform where I built React dashboard workflows and Flask APIs that converted AI and vision-model outputs into a clear user experience.

What attracts me to Voxel is the company's goal of making industrial workplaces safer through proactive detection. Instead of only reviewing incidents after they happen, Voxel uses AI and computer vision to identify risks early, like PPE issues, forklift collision risks, blocked exits, or unsafe patterns, so teams can act before someone gets hurt. That mission really connects with me because this role combines what I enjoy building — dashboards, APIs, data flows, and application features — with real-world safety impact.

Work Experience

USC Dornsife — Software Engineer

You built a **multi-tenant internal platform** for 500+ users with backend services, dashboards, Azure SQL, row-level security, and normalized data models. Your main work was improving application performance, building secure APIs, handling async jobs, and debugging data issues across the system.

Say in interview:
"I worked on full-stack features and backend services for a secure internal platform, mainly around APIs, Azure SQL, row-level authorization, dashboard performance, and async job orchestration."

Grad8 — Software Engineer Intern

You built parts of an **AI grading platform** where instructors could upload assignments and receive AI-assisted grading feedback. You worked on React dashboards, Flask APIs, OCR/multimodal LLM outputs, provider switching, batch processing, and debugging issues like missing or mislabeled AI outputs.

Say in interview:
"I helped build the application layer for an AI-powered grading product, turning multimodal LLM and vision outputs into usable dashboard workflows for instructors."

USC AD Labs — ML Research Assistant

You worked on **autonomous-driving/trajectory-planning research**, mainly interpreting computer-vision/perception outputs and building pipelines over large trajectory datasets. Your work connected AI model outputs, HD-map geometry, attention-based encoders, and evaluation pipelines to improve planning accuracy and find edge cases.

Say in interview:
“I worked with perception and trajectory data, building pipelines and model-integration workflows that helped convert CV/AI outputs into downstream planning signals.”

Aasma Technology Solutions / VibrantChem — Full Stack Developer Intern

You built a **MERN-based operational dashboard/CRM system** with REST APIs, JWT authentication, role-based access, microservices, WebSockets, Docker/Jenkins CI/CD, and monitoring. The system handled large record syncing and real-time dashboard updates for business operations.

Say in interview:
“I built full-stack MERN features, secure APIs, real-time WebSocket updates, and CI/CD workflows for an operational dashboard used to manage large-scale business data.”

LST Capitals — Web Developer Intern

You worked on a **cross-platform booking/transaction system** using React/React Native, Flask APIs, Firebase, authentication, payment workflows, and real-time synchronization. Your main focus was backend APIs, booking records, real-time updates, and secure transaction/webhook flows.

Say in interview:
“I built React/React Native client features and Flask APIs for a booking and transaction platform, including real-time Firebase sync, token-based auth, and payment/webhook workflows.”

Projects

PiXMod — Next.js, CLIP, SAM, Stable Diffusion

You built an **AI computer-vision SaaS product** for image segmentation, semantic search, and AI-powered image workflows. It used SAM for segmentation, CLIP for embeddings/search, Stable Diffusion for generation/editing, a Next.js dashboard, and Stripe subscriptions.

Say in interview:
“PiXMod was an AI SaaS platform where I built a Next.js dashboard around computer-vision workflows like segmentation, semantic search, embeddings, and subscription-based access.”

FinNexus — Next.js, TypeScript, Kubernetes, WebSockets

You built a **real-time fintech analytics platform** for transaction data, dashboards, secure access, and live updates. The project focused on high-volume data, token-based access control, WebSockets for real-time updates, and Kubernetes-backed deployment/CI/CD.

Say in interview:
“FinNexus was a full-stack fintech analytics platform where I built real-time dashboards, secure transaction workflows, WebSocket updates, and scalable deployment using Kubernetes.”

HMIS — MERN, TypeScript, Sentry

You built an **enterprise healthcare management dashboard** with patient/appointment workflows, consent-based access, alerts, and monitoring. It used MERN, TypeScript, Twilio alerts, Sentry monitoring, and role/consent-based access to reduce manual administrative work.

Say in interview:
“HMIS was a healthcare dashboard where I built appointment workflows, consent-based access, admin dashboards, Twilio alerts, and monitoring with Sentry.”

Why Voxel?

- I’m drawn to Voxel because the mission feels very practical and real.
- Instead of only reviewing safety incidents after someone gets hurt, Voxel uses AI and computer vision to detect risks earlier.
- I like that the work connects software engineering with real-world safety impact.
- For me, that is exciting because I enjoy building dashboards, APIs, auth flows, and data systems that turn complex signals into something useful for users.

- Voxel feels like the right place where my full-stack background and interest in AI/CV can come together meaningfully.

What do you know about what we do?

- Voxel is an AI/computer vision company focused on workplace safety, operations, and risk in industrial environments.
- The platform uses existing camera infrastructure and turns video into actionable insights.
- It helps detect things like PPE issues, forklift or vehicle hazards, blocked exits, spills, unsafe behaviors, and operational bottlenecks.
- The bigger idea is moving safety from reactive to proactive.
- I also saw that Voxel works with industries like warehousing, manufacturing, logistics, retail distribution, and supply chain.

Why am I a perfect fit?

- I think I'm a strong fit because this role sits exactly at the layer I enjoy: applications, APIs, dashboards, data flows, and user-facing workflows.
- At USC Dornsife, I built backend services, Azure SQL data models, row-level authorization, async jobs, and dashboard features for 500+ users.
- At Grad8, I worked on React and Flask workflows that turned AI and vision-model outputs into a clear user experience.
- That maps well to Voxel because the role needs someone who can build reliable product features around AI/CV safety signals.

---

§4 — HARDEST FEEDBACK ⭐ (also: "what are you working on now")

LLM playground · current role

Landscape

- At USC Dornsife I've been building an LLM playground — submit a prompt, it runs against a baseline model, then an orchestrator runs the output through judge models to evaluate it
- I own the orchestration layer, the logging, and the job format

Situation

- First working version did the obvious thing: submit → baseline → orchestrator → judges → render everything at once
- Technically correct. Results accurate, logs clean
- My manager tried it and said bluntly it was a bad experience — the user just sits there waiting for the whole pipeline
- **My first instinct was to defend it.** The models genuinely take that long. I thought this was a model latency problem, not a design problem

Action

- Once I stopped being defensive, I realized he wasn't saying the pipeline was slow. He was saying **I'd designed the whole thing around the process being correct and hadn't thought about what a person experiences while they wait**
- I'd been treating latency as a number to reduce. He was pointing out it's time a human sits through. Different problems — I'd only solved one
- Fix wasn't hard once I saw it: stream the baseline answer as it generates instead of holding it until complete
- **Then the realization that made it work:** while the user reads the baseline, that's dead time I was throwing away. So kick off the orchestrator and judge models in parallel, in the background, during the read
- By the time they finish reading the baseline, the evaluation is usually already there

Result

- Active users went up — stopped feeling like something you submit to and walk away from
- **Total time barely changed. Perceived time changed completely**
- **The habit that stuck:** now when I design anything with a wait in it, the first question I ask is what the user sees *during* the wait, before I ask how to make it faster

Follow-up ammo

- *How did you take it?* → "Not gracefully. I explained why the models take that long, which was answering a question he hadn't asked."
- *Was he right?* → "Completely. And the uncomfortable part is the fix was maybe half a day of work — it wasn't hard, I just hadn't looked at it from the user's side."

---

§5 — HARDEST TECHNICAL PROBLEM

Aasma · monolith → phased microservices

Landscape

- Full-stack intern at Aasma on a MERN resource management platform — inventory sync, dealer payment reconciliation, raw material tracking. Single monolith, Scrum, deadlines locked
- Midway through the project, so any architectural change carried real delivery risk

Situation

- Once we deployed more frequently: if one feature broke — say inventory sync — we'd roll back the entire application
- Rollbacks were slow, debugging took hours, and a mistake in one area could take down unrelated things like payments or reporting

Action

- Broke it into two questions: why is debugging so painful, and what level of change is realistic given the timeline
- Two real options — stay monolithic with stricter testing and release discipline, or move toward services mid-project
- **I proposed phased, not a rewrite.** Carve out the high-risk, high-change areas first — inventory sync, dealer payments, raw material tracking. New features built as services immediately, old components migrated during sprint slack
- Trade-off was upfront complexity vs. long-term stability. Incremental gave us isolation without blowing the timeline

Result

- Failures became easy to reason about — inventory sync breaking no longer took payments down
- Debugging went from hours to focused log tracing in one service. Rollbacks became scoped instead of system-wide
- Releases felt predictable again
- Lesson: architecture choices directly affect developer velocity and reliability, not just design purity

Follow-up ammo

- *Risky time to suggest it — why were you confident?* → "It was a pattern, not one incident. Every fix was simple; finding *where* it broke took the time. That told me the problem was coupling, not bugs."
- *How, as an intern, without overstepping?* → "I didn't say 'let's rewrite everything.' I walked them through one incident and proposed extracting *one* service as an experiment. Concrete and low-risk."
- *What did it introduce?* → "Operational complexity — inter-service comms, config, pipelines. Acceptable because failures became local and loud instead of silent and systemic."

---

§6 — HARDEST BUG / DEBUGGING

Aasma · silent partial failures

⚠️ Same system as §5 — don't tell both

Landscape

- Same Aasma platform, still monolithic, deploying frequently under tight sprints

Situation

- Certain deployments caused partial system failures — inventory updates would stop reflecting correctly while reporting still looked fine
- **Nothing fully crashed.** From outside the system looked up, but data was silently inconsistent
- Debugging took hours, rollbacks were painful because they hit the whole application

Action

- Tried to narrow the blast radius mentally before touching code
- Logs showed failures consistently followed deployments touching inventory and payment logic — but errors were getting swallowed because everything shared one runtime and logging context
- Added granular logging around the critical flows, then correlated timestamps between user actions, API calls, and database writes
- Pattern emerged: a failure in one async operation would block shared resources and leave the system partially updated, **without triggering a hard failure**
- Root cause wasn't a single bug — it was tight coupling, where one failing path indirectly affected unrelated features

Result

- Short term: made failure states explicit so partial updates couldn't silently succeed
- Long term: this directly informed isolating high-risk components into services
- **Biggest takeaway: good debugging isn't just fixing bugs — it's making failure modes obvious and contained**

Follow-up ammo

- *Why was it hard to catch in testing?* → "Tests covered individual feature correctness, not failure interaction. In isolation everything worked. It only surfaced when async operations overlapped under real usage."
- *How did you know it wasn't a one-off?* → "The repetition. Different features broke, same failure shape — partial updates, no crash, inconsistent state. When symptoms share a shape, the structure is the problem."
- *Alone or with others?* → "Investigated solo to cut noise, then walked the team through the pattern once I had evidence. That aligned everyone on root cause instead of debating symptoms."

---

§7 — DISAGREEMENT WITH A TEAMMATE

Aasma · CI/CD testing strategy

Landscape

- Aasma, cross-platform data and resource management system, aggressive timeline, pressure to move fast

Situation

- Some teammates felt extensive automated tests were slowing development and wanted to cut coverage to ship faster
- I was concerned that given how coupled the system was, fewer tests meant more rollback risk and harder debugging — something we already struggled with

Action

- Instead of debating opinions, I reframed it as a trade-off and proposed treating it as an experiment
- Built a comparison: one pipeline with reduced tests, one with targeted tests on high-risk areas. Measured build time, failure rates, debugging effort across a few sprints
- **Was deliberate about listening.** The speed concern was valid, so I made sure the solution addressed it instead of dismissing it. The goal wasn't to win the argument

Result

- Hybrid: lightweight tests every commit, deeper tests nightly
- Less friction during development, still caught issues early

- The discussion set a tone — disagreements became about evidence and outcomes, not preferences

Follow-up ammo

- *What made it hard?* → "Both sides were optimizing for something reasonable. If I'd framed it as one side being wrong, it would've stalled."
- *Did you compromise on anything?* → "Yes — I'd leaned toward broad coverage everywhere. I came around to the idea that not all parts carry the same risk." *(→ this is §8)*
- *If they hadn't agreed?* → "I'd have accepted it and treated it as a data point. If issues increased, the conversation reopens naturally. Influencing isn't always getting your way."

---

§8 — CHANGED YOUR MIND ⭐

Aasma · CI/CD test coverage

> ⚠️ Same situation as §7. If you've used it: *"I touched on this earlier, but there's a specific moment I can go deeper on."*

Landscape

- Aasma, aggressive timeline, pressure to ship

Situation

- Teammates wanted to cut test coverage to move faster. I pushed back firmly — the system was tightly coupled, thinner tests meant more rollbacks and harder debugging
- **My position was that coverage should be broad, everywhere.** Felt like the obviously responsible answer and I was confident about it

Action

- What changed my mind wasn't an argument. It was someone asking **which specific tests had caught anything real in the last few sprints**
- **I didn't have a good answer.** When I looked, a lot of coverage sat on parts that barely changed and had no external dependencies — stable code we tested thoroughly out of habit, not risk
- Meanwhile the things that actually broke — inventory sync, dealer payments — had real coupling and external dependencies
- **That reframed it: I'd been treating coverage as one dial where more is better. It isn't — it's a budget you allocate by risk**

- And the part I got wrong about people: **I'd read their pushback as impatience about deadlines. It wasn't. They were optimizing for something real and I hadn't engaged with it**
- So we ran the experiment rather than kept arguing, and landed on the hybrid

Result

- Development friction dropped, issues still caught early
- **What changed in me: I stopped thinking about coverage as a percentage and started thinking about it as risk allocation.** Compromise didn't mean lowering the standard — it meant applying it where it bought something

Follow-up ammo

- *Why did that question land?* → "Because it was answerable and I couldn't answer it. Before that we were trading positions, and both sound reasonable in the abstract."
- *Did it feel like losing?* → "A bit. But I was defending a principle rather than an outcome. Once I saw the coverage wasn't protecting anything at risk, defending it was ego."
- *How do you judge risk now?* → "How often does it change, does it touch anything external, what's the blast radius. Better filter than a coverage percentage."

---

§9 — EXPLAINING TO A NON-TECHNICAL PERSON ⭐

Grad8 · autonomous grading system

Landscape

- At Grad8 I designed an autonomous grading system — AI grading against a consistent answering standard set by the educator, evaluating each student submission and generating a report per person

Situation

- My manager was from a non-technical background, and she needed to understand how the model evaluated every submission and produced those reports
- **I tried explaining it verbally first and it wasn't landing.** She was nodding, but the questions weren't getting more specific — which is usually the tell

Action

- Realized a walkthrough of the system flow would do more than description, but even that was still me explaining a *process*
- So I built a presentation **plus a working prototype with hardcoded reports for three students**
- Then I walked the flow and showed the prototype at the same time — here's what the report says about this student, and here's the step that produced it
- **The thing that made it work: I showed the output before the process.** That's the reverse of how I'd naturally explain it, but it's the order she could actually follow

Result

- She got confidence in how the pipeline was designed — and more usefully, she could finally give real input, [what she caught / flagged], which I'd have missed
- Delivered with the clarity we'd been asked for
- **Lesson: with a non-technical audience, showing the output first and the process second works far better than the order that feels natural to me**

Voxel bridge: "That's the part of Actions I find interesting — same problem. A safety manager doesn't want a confidence score, they want to know whether to send someone to aisle four."

---

§10 — CROSS-FUNCTIONAL

LST Capitals · API shape

Landscape

- LST Capitals, ~2 months, cross-platform booking app for household services — cleaning, AC repair, childcare
- I owned the Flask backend. Separate frontend team on React Native
- **First time I'd worked with a dedicated frontend team rather than building both sides myself**

Situation

- I designed the APIs the way the data was modeled — clean, normalized, one resource per endpoint. Bookings, services, providers, slots. Made total sense from where I sat
- Frontend came back saying one booking screen took four sequential calls. On mobile, on the connections our users had, that was seconds of loading spinners
- **My first reaction was that this was a caching problem** — fetch once and hold it
- Their answer: slot availability changes constantly, and stale availability means double-bookings. **Fair point I hadn't thought about, because I wasn't looking at the screen — I was looking at the schema**

Action

- Instead of arguing abstractly, I asked one of them to walk me through a single screen call by call, and what the user sees during each
- **That's when it clicked: I'd designed around the data model, they needed it organized around screens**
- Added composite endpoints for the high-traffic screens — booking detail, slot picker — keeping the granular ones underneath for everything else
- Then made it a process, not a one-off: before building any endpoint, write the response shape and share it first. **Changing a JSON sketch is free. Changing a shipped endpoint wired into three screens costs a day**
- Set up regular calls — not status updates, but "here's what I'm about to build, does this shape work"
- Also agreed an error format they could show users directly, instead of technical messages they'd translate

Result

- Booking screen went from four calls to one
- Integration got much less painful — most endpoints worked first try because the shape was agreed before I wrote a line
- Got in front of the client ~3 weeks early, mostly from not losing days to integration rework
- **Lesson: I'd been treating the API as the backend's output. It's a shared interface, and the consuming team has constraints I can't see from my side**

Voxel bridge: "That's why the perception-to-applications handoff here is interesting — same shape. One team produces output, another has to make it usable, and the constraints on the consuming side aren't visible unless you go ask."

---

§11 — OWNERSHIP / INITIATIVE

Grad8 · reliability

Landscape

- Grad8, AI-powered grading platform used by instructors at scale
- My formal role was backend feature development. But as usage grew it started behaving less like a prototype and more like a production product

Situation

- When grading failed or behaved inconsistently, **there was no clear owner for diagnosing it end to end**
- Bugs got fixed locally but patterns repeated. That gap wasn't in anyone's job description, and it was affecting instructor trust — a bigger problem than any single feature

Action

- Mapped the full grading flow — upload → OCR → LLM grading → result storage — and found where failures repeated
- Introduced structured error handling across services, better logging around batch jobs, basic failure categorization so issues were triageable
- **Took responsibility when things broke** instead of treating it as someone else's layer — followed issues through to resolution even when it meant coordinating across frontend, backend, and infra

Result

- Grading failures dropped, debugging time went down, instructors stopped reporting inconsistent behavior as often
- Team started thinking more end-to-end about quality
- **Ownership isn't about title — it's noticing what's missing and carrying it, even when it's uncomfortable**

Follow-up ammo

- *Did you make a mistake owning it?* → "Yes — I tried to fix too much at once and slowed myself down. Raising the bar didn't mean perfecting everything, it meant finding the few changes that unlocked clarity."
- *How did you avoid stepping on toes?* → "Framed everything around outcomes, not fixing people's mistakes. I'd bring the repeated failure pattern and ask 'would it help if we standardized this?'"

---

§12 — A MISTAKE YOU MADE

CaptionCraft

> ⚠️ Not on your resume — open with *"this was a side project"*

Landscape

- CaptionCraft, an AI image captioning project. Self-driven, so I owned everything — data, modeling, evaluation, deployment

- Experimented with BLIP, GIT, ViT-GPT-2, plus a custom model

Situation

- **My mistake: I assumed model architecture alone would drive accuracy**
- Spent a lot of time swapping models and tuning hyperparameters. Performance plateaued around 81% and I couldn't explain why increasingly complex models gave marginal gains

Action

- Stepped back and analyzed failure cases instead of chasing metrics
- Many incorrect captions weren't model capacity — they were **dataset bias and noisy labels. I'd treated the dataset as fixed truth, which was the core mistake**
- Cleaned and restructured the dataset, balanced captions across contexts, added prompt refinement and comparative evaluation
- Built a more granular evaluation setup so I could understand *why* a caption was wrong, not just that it was

Result

- 81% → ~96%, without increasing model complexity
- **Changed how I learn: I now question my assumptions about data quality, evaluation, and problem framing before reaching for more advanced tools**
- Carried into later work — I spend more time understanding failure modes before optimizing

Follow-up ammo

- *How did you know it was your approach, not a hard problem?* → "Diminishing returns. Every change was expensive in effort, cheap in impact. When complexity rises but understanding doesn't, the bottleneck isn't where you think."

---

§13 — AMBIGUITY / UNCLEAR REQUIREMENTS

Big-Brain

⚠️ Not on your resume — open with *"this was a side project"*

Landscape

- Big-Brain, an AI document intelligence platform. No well-defined product spec — we knew the goal (ingest documents, allow intelligent querying) but there was no success definition, no fixed feature list, and users expected different things
- The ambiguity was conceptual, not just technical

Situation

- I owned figuring out what to build first without waiting for perfect clarity. Easy to either overbuild or freeze

Action

- **Separated unknowns from decisions that couldn't wait.** We didn't know the final UI, but we knew ingestion and retrieval accuracy were non-negotiable
- **Asked clarifying questions in terms of outcomes, not features.** Instead of "should we support PDFs or Word," I asked "what would make a user trust an answer from this system?" That anchored decisions on reliability
- **Made assumptions deliberately and documented them.** Assumed search accuracy mattered more than speed initially, so prioritized vector indexing over UI polish. Treated assumptions as things to validate, not defend

Result

- Shipped a focused MVP — 50K+ documents, ~95% query accuracy, real user feedback early
- Flexible enough that when requirements shifted (collaborative features), we didn't undo major decisions
- **Handling ambiguity isn't guessing right — it's creating momentum while keeping decisions reversible**

Follow-up ammo

- *Which assumptions were safe?* → "I looked at reversibility. Vector search was safe — indexes can be swapped. Hard-coding workflows would've been much harder to undo."
- *Was an assumption wrong?* → "Yes — we assumed single-user was enough. Once people collaborated, that broke. Because retrieval wasn't coupled to user state, we could add it without reworking the core."

---

§14 — SYSTEM DESIGN / SCALE

LST Capitals · bookings + payments

Landscape

- LST Capitals, backend for a cross-platform house-help booking platform. Real-time bookings, worker availability, payments through a UAE gateway
- User base wasn't huge, but bookings and payments are very sensitive to failure

Situation

- I owned backend APIs for bookings, payments, real-time status
- Challenge: balancing responsiveness with consistency. Users expect instant confirmation, but payments involve async callbacks, retries, failures. Naive design either blocks users waiting on payment or risks inconsistent state

Action

- **Made booking optimistic but state-driven** — a request immediately creates a booking in `pending` so the user gets fast feedback
- Payments handled asynchronously via gateway webhooks. **Booking state transitions only on authoritative payment events**
- Trade-off: complexity vs. reliability. Async made it harder to reason about, but kept UX fast without pretending payments are instant
- **Made booking and payment flows idempotent** so retries or duplicate webhooks couldn't corrupt state

Result

- Handled growing volume without blocking users or risking double charges
- Failures became recoverable, not catastrophic — a delayed callback meant the system eventually converged
- **Early lesson in distributed systems: fast responses don't require immediate consistency, as long as state transitions are explicit and reliable**

Follow-up ammo

- *Latency vs consistency?* → "Synchronous would be simpler to reason about but UX suffers from unpredictable gateway latency. We accepted temporary inconsistency — booking shows pending — in exchange for responsiveness. Key was making it visible and short-lived, not hidden."
- *Failure modes you designed for?* → "Webhook retries, partial failures, duplicate events. Assumed callbacks could arrive late or twice, so every handler was defensive. Turned potential incidents into routine transitions."
- *Observability?* → "Structured logging around booking IDs and transaction IDs so we could trace a request across services and callbacks — answering 'why is this still pending' without inspecting the database."

---

§15 — PRIVACY / ETHICS ⭐

HMIS · consent-first design

Landscape

- HMIS, a hospital management system built during a hackathon. Patient records, appointments, consent-based access to medical history via India's Ayushman Bharat Digital Mission APIs
- Hackathon project, but real healthcare data — which raised the bar

Situation

- We realized we could move much faster by giving doctors broad access to records by default. Simpler flows, easier demos
- **But medical data is sensitive, and developer convenience isn't the same as what's right for patients**
- Nobody was telling us not to. That's what made it hard

Action

- **Pushed for consent-first**, even though it added complexity and slowed us down
- Instead of default access, patients explicitly granted consent before doctors could see historical records
- Limited data exposure to what was necessary for a given interaction rather than pulling full histories every time
- **Careful how I raised it** — not "this is wrong," but "if I were a patient, how would I expect my data handled?" Kept it grounded and respectful under time pressure

Result

- Slightly more complex, but aligned with real healthcare expectations. Doctors got what they needed, patients kept control
- **Building responsibly often means the harder path even when shortcuts exist**
- Reduced administrative workload ~30% while keeping the consent gate

The Voxel bridge — use this:

> "You're putting AI on cameras pointed at people at work, and your position is 'visibility into hazards, not insights into individuals.' I think that's the right line. Practically I'd read that as storing the detection and a bounding box, not an identity. Alerts that say 'a worker,' not a name. Short retention, role-based access on clips.

> And it's not only ethics — it's adoption. A safety tool that starts feeling like surveillance loses worker trust, and then people work around it, avoid the camera, stop reporting near-misses. **The privacy design is the adoption design.**"

Follow-up ammo

- *Why did ethics matter in a hackathon?* → "Prototypes shape how real systems get built. If you normalize shortcuts early they carry forward."
- *Pushback on time constraints?* → "Yes. I acknowledged the trade-off openly, but framing it around user trust helped everyone see it wasn't optional complexity — it was part of the problem we were solving."

---

§16 — QUESTIONS FOR THEM

Ask 2–3. Not asking is a fail.

- **"What's the hardest part of the product that isn't the ML?"** *(likely alert fatigue or edge fleet ops — their answer tells you a lot)*
- **"How do you measure whether an alert was actually useful?"** *(the real product problem)*
- **"How does perception hand off to the applications team — clean interface, or messier than the org chart suggests?"** *(you've lived this)*
- "What's a decision the team made that you disagreed with?"
- "What separates someone who does well here from someone who struggles?"
- "What did you ship last sprint?"